

Ideal AI 홈페이지 Blog 화면

화면의 사용자 노출 원문 전체를 추출한 챗봇 SDK 적재용 PDF

페이지 소스의 텍스트, 메타데이터, 카드·리스트·통계·FAQ·블로그 본문을 자동 추출했습니다.

원문 출처: app/blog/page.js

INSIGHTS & KNOWLEDGE

AI 실무자가 쓰는

진짜 기술 블로그

마케팅 글이 아닙니다. 프로젝트에서 실제로 부딪힌 문제, 실험 결과, 실패 경험. 현장에서 배운 것을 그대로 씁니다.

STAY UPDATED

AI 실무 인사이트를

가장 먼저 받아보세요.

뉴스레터 구독

Blog: AI 실무자가 쓰는 진짜 기술 블로그

마케팅 글이 아닙니다. RAG, LLM, AI Agent, 업무 자동화 프로젝트에서 실제로 부딪힌 문제, 실험 결과, 실패 경험. 현장에서 배운 것을 그대로 씁니다.

원문 출처: lib/posts.js

Hybrid RAG가 단순 벡터 검색보다 나은 이유: 실전 비교 실험

키워드 검색과 시맨틱 검색을 합치면 왜 더 나은가? 3가지 실험 케이스와 수치로 증명합니다. 기업 내부 문서 검색에서 Hybrid RAG가 정확도 23%p 향상을 기록한 이유.

8분

장은선

기업 내부 문서 검색 프로젝트를 진행하면서 단순 벡터 검색 대비 Hybrid RAG가 훨씬 낫다는 결론을 내렸습니다. 직접 실험한 데이터를 공개합니다.

왜 벡터 검색만으로는 부족한가

시맨틱 검색(벡터 검색)은 의미적으로 유사한 문서를 잘 찾습니다. "퇴직금 지급 기준"을 검색하면 "퇴직급여"나 "근속연수 산정" 같은 연관 문서를 잘 잡아냅니다. 하지만 정확한 조항 번호나 특정 코드명을 검색할 때는 오히려 정확도가 떨어집니다. "제23조 제4항"을 찾아야 하는데 의미 유사도 기반으로 엉뚱한 조항이 올라오는 상황이 실제로 자주 발생했습니다.

실험 설계

대형 금융사 내규 문서 3,200페이지를 대상으로 세 가지 검색 방식을 비교했습니다. 순수 BM25(키워드), 순수 Dense Retrieval(벡터), 그리고 두 방식을 결합한 Hybrid(RRF 앙상블). 평가 질문은 실제 준법감시팀 직원들이 자주 묻는 질문 200개를 사용했고, 정답 문서는 사전에 수동으로 레이블링했습니다.

실험 결과 요약

BM25 (키워드)

Top-5 Recall

Dense Retrieval

Hybrid RAG (RRF)

Top-5 Recall +23%p

RRF(Reciprocal Rank Fusion) 적용 방법

RRF는 두 검색 결과의 순위를 결합하는 간단하지만 강력한 방법입니다. 각 검색 방식에서 나온 순위를 $1/(rank + k)$ 공식으로 점수화하고 합산합니다. $k=60$ 이 일반적으로 잘 작동합니다. 복잡한 가중치 튜닝 없이 안정적인 성능을 냅니다.

어떤 경우에 어떤 방식이 유리한가

조항 번호, 코드명, 고유명사 검색: BM25가 압도적. 개념적 질문("임원 보수 관련 규정은?"): Dense Retrieval이 강함. 혼재된 실제 업무 질문: Hybrid가 일관되게 가장 좋음. 결론적으로 예측 불가능한 실제 사용자 쿼리에는 Hybrid를 기본값으로 사용하는 것이 안전합니다.

GPT-4o vs Claude 3.5: 실무 관점에서 어떤 모델을 선택해야 하나

벤치마크가 아닌 실제 업무 케이스(법률 문서 분석, 코드 생성, 한국어 처리)에서 두 모델을 직접 비교했습니다. 각각 어떤 상황에서 우위를 보이는지 정리합니다.

12분

홍유정

벤치마크 점수는 참고만 하고, 실제 프로젝트에서 두 모델을 교차 테스트한 결과를 솔직하게 공유합니다.

테스트 환경

3개 도메인(법률 문서 분석, Python 코드 생성, 한국어 문서 작성)에서 각각 50개 태스크를 실행했습니다. 온도는 0으로 고정해 재현성을 확보했고, 동일한 시스템 프롬프트와 few-shot 예시를 양쪽에 제공했습니다.

법률 문서 분석: Claude 우위

Claude 3.5 Sonnet이 긴 문서 처리와 정확한 조항 인용에서 더 나은 성능을 보였습니다. GPT-4o는 간혹 없는 조항을 만들어내는(hallucination) 경우가 있었고, Claude는 "해당 내용이 문서에 없습니다"라고 명확하게 답하는 경향이 강했습니다. 리스크가 높은 업무일수록 Claude가 더 안전합니다.

코드 생성: GPT-4o 우위

Python 코드 생성에서는 GPT-4o가 전반적으로 더 빠르고 실용적인 코드를 냅니다. Claude도 코드 품질은 높지만, 설명이 길어지는 경향이 있어 바로 사용할 수 있는 코드 snippet을 원할 때는 GPT-4o가 편했습니다.

한국어 처리: 거의 동등, 뉘앙스 차이

문법적 정확도는 비슷하지만 문체가 다릅니다. Claude는 더 격식체에 가깝고, GPT-4o는 조금 더 자연스럽고 구어적입니다. B2B 문서 작성에는 Claude, SNS나 마케팅 카피에는 GPT-4o가 더 잘 맞다는 인상을 받았습니다.

n8n으로 만드는 AI 문서 자동화 워크플로우: 코딩 없이 구현

이메일로 들어오는 계약서를 자동 분류하고, GPT로 리스크 조항을 추출한 뒤 Slack으로 알림을 보내는 파이프라인을 n8n으로 구현하는 방법을 단계별로 설명합니다.

10분

코드 한 줄 없이 AI 기반 계약서 분석 파이프라인을 만들 수 있습니다. n8n + GPT-4o 조합으로 실제 구현한 사례를 단계별로 공개합니다.

전체 파이프라인 개요

Gmail에서 계약서 첨부 이메일 감지 → PDF 텍스트 추출 → GPT-4o로 리스크 조항 분석 → Google Sheets에 기록 → Slack 알림 발송. 이 전체 흐름이 n8n 워크플로우 하나로 돌아갑니다.

핵심 노드 설정

Gmail Trigger 노드: 특정 라벨(계약서)이 붙은 메일만 트리거. HTTP Request 노드: PDF를 받아 텍스트 추출 API 호출. OpenAI 노드: 시스템 프롬프트로 리스크 조항 추출 지시. 응답은 JSON 구조로 받아 Sheets에 컬럼별 저장합니다.

실제로 쓰는 GPT 프롬프트

리스크 추출 프롬프트는 역할 정의, 출력 형식 지정, few-shot 예시 순서로 구성합니다. 출력을 JSON으로 강제하면 다음 노드에서 파싱이 편합니다. "response_format: json_object"를 API 파라미터로 설정하는 것을 잊지 마세요.

운영 시 주의점

n8n 클라우드 vs 셀프호스팅 선택이 중요합니다. 계약서 같은 민감 문서는 셀프호스팅 필수. Execution 로그 보존 기간도 확인하세요. GPT API 비용은 계약서 1건당 약 \$0.01~0.03 수준으로 충분히 합리적입니다.

Case Study

은행 내규 챗봇 구축기: 3,200페이지를 RAG로 처리한 6주의 기록

대형 은행 준법감시팀의 내규 문서 챗봇 프로젝트 전 과정을 공개합니다. 청킹 전략, 임베딩 모델 선택, 평가 지표 설정까지. 실제로 부딪힌 문제와 해결책을 솔직하게 씁니다.

15분

6,000명이 사용하는 대형 은행 준법감시팀 내규 챗봇 구축 전 과정을 공개합니다. 성공한 것만큼 실패한 시도도 솔직하게 씁니다.

프로젝트 배경

준법감시팀 직원 1인이 하루 평균 수십 건의 내규 문의를 처리하고 있었습니다. 문서가 3,200페이지, 최근 3년간 개정 이력까지 포함하면 사실상 전문가도 즉시 답하기 어려운 수준. AI로 대체할 수 있다면 담당자가 진짜 필요한 심층 업무에 집중할 수 있습니다.

청킹 전략이 성패를 갈랐다

첫 시도는 고정 길이 청킹(512토큰). 결과는 참혹했습니다. 조항이 중간에 잘리면서 맥락이 끊기고, 검색 품질이 낮았습니다. 두 번째 시도로 구조 기반 청킹(조, 항, 호 단위)을 적용했고 정확도가 크게 올랐습니다. 법률/규정 문서는 반드시 문서 구조를 파악하고 청킹해야 합니다.

임베딩 모델 선택

OpenAI text-embedding-3-large vs bge-m3 (한국어 특화) 비교 결과, 한국어 법률 문서에서는 bge-m3가 10%p 이상 높은 검색 성능을 보였습니다. 비용도 셀프호스팅 기준으로 훨씬 저렴합니다. 한국어 특화 문서라면 한국어 임베딩 모델을 반드시 검토하세요.

최종 성과

조회 시간

78% 단축

평균 4시간 → 즉시 답변

만족도

준법감시팀 내부 평가

처리 건수

3배 증가

같은 인원으로

LoRA 파인튜닝 실전 가이드: 도메인 특화 LLM을 최소 비용으로

A100 한 장, 48시간, 50만 원 이내의 예산으로 법률 도메인 특화 LLM을 파인튜닝한 과정을 공유합니다. 하이퍼파라미터 선택, 데이터 품질 관리, 평가까지.

14분

전체 모델 파인튜닝은 수천만 원이 드는 일이지만, LoRA를 쓰면 A100 1장, 48시간, 50만 원 이내로 같은 효과를 낼 수 있습니다.

LoRA가 왜 효율적인가

LoRA(Low-Rank Adaptation)는 모델 전체 파라미터를 학습하는 대신, 저랭크 행렬 두 개를 삽입해서 그것만 학습합니다. 전체 파라미터의 0.1~1% 수준만 학습하므로 메모리와 연산이 극적으로 줄어듭니다. 원본 모델 가중치는 건드리지 않으므로 언제든지 원래 상태로 되돌릴 수 있다는 것도 장점입니다.

실습 환경 설정

Lambda Labs에서 A100 80GB 인스턴스를 시간당 약 \$1.2에 빌렸습니다. 48시간 기준 약 \$57, 한화 약 8만 원. 베이스 모델은 Llama 3 8B, 라이브러리는 Hugging Face PEFT + TRL을 사용했습니다. QLoRA를 적용해 4비트 양자화로 메모리를 더 아꼈습니다.

데이터 품질이 전부다

법률 Q&A 5,000쌍을 만들었는데, 초기 데이터 1,000개로 먼저 학습시키고 오답 케이스를 분석해 나머지 4,000개를 보강했습니다. 데이터 양보다 품질이 훨씬 중요합니다. 잘못된 정보가 포함된 학습 데이터는 모델을 망칩니다.

핵심 하이퍼파라미터

rank=16, alpha=32, dropout=0.1, learning_rate=2e-4, batch_size=4(gradient_accumulation=4)로 effective batch=16. 에포크는 3회. 이 조합이 저희 태스크에서 가장 안정적이었습니다. rank는 태스크 복잡도에 따라 4~64 사이에서 실험해보세요.

RAG 시스템 평가하기: RAGAS 지표를 실무에 어떻게 적용할까

Faithfulness, Answer Relevancy, Context Precision. RAG 평가 지표들이 실제로 무엇을 측정하는지, 그리고 프로젝트에서 어떤 지표를 우선시해야 하는지 정리합니다.

11분

RAG 시스템을 만들었으면 제대로 평가해야 합니다. 막연히 "잘 되는 것 같다"는 직관보다 RAGAS 지표로 수치화하세요.

RAGAS가 측정하는 것

Faithfulness: 생성된 답변이 검색된 컨텍스트에 얼마나 충실한가 (hallucination 감지). Answer Relevancy: 답변이 질문에 얼마나 관련성 있는가. Context Precision: 검색된 문서 중 실제로 답변에 사용된 비율. Context Recall: 정답 도출에 필요한 문서가 검색됐는가.

어떤 지표를 먼저 봐야 하나

리스크가 높은 도메인(금융, 법률, 의료): Faithfulness 최우선. 환각이 가장 위험하기 때문입니다. 일반 Q&A 챗봇: Answer Relevancy와 Context Recall 균형. 검색 최적화가 목표라면: Context Precision에 집중해 불필요한 문서가 섞이지 않도록.

실무 적용시 함정

RAGAS는 평가 자체에도 LLM을 사용합니다. 평가용 모델이 약하면 평가 결과도 신뢰하기 어렵습니다. GPT-4o 이상을 평가자로 쓰는 것을 권장합니다. 또한 테스트셋을 실제 사용자 쿼리로 구성해야 의미있는 평가가 됩니다.

LangGraph로 Multi-Agent 시스템 설계하기: 패턴 3가지

Supervisor, Parallel, Sequential. LangGraph에서 자주 쓰는 멀티에이전트 패턴 3가지를 실제 코드와 함께 설명합니다. 어떤 업무 흐름에 어떤 패턴이 맞는지.

13분

LangGraph로 멀티에이전트 시스템을 구축할 때 매번 새로 설계하지 않아도 됩니다. 검증된 패턴 3가지를 알면 대부분의 업무 자동화에 대응할 수 있습니다.

패턴 1: Supervisor 패턴

중앙 관리자(Supervisor) 에이전트가 작업을 분배하고 결과를 취합합니다. 복잡한 태스크를 여러 전문 에이전트에 위임할 때 적합합니다. 예: 보고서 생성 에이전트가 데이터 수집 에이전트, 분석 에이전트, 작성 에이전트를 지휘하는 구조.

패턴 2: Parallel 패턴

독립적인 여러 에이전트가 동시에 실행되고 결과를 합칩니다. 서로 의존성이 없는 병렬 작업에 적합합니다. 예: 여러 데이터 소스(뉴스, 공시, SNS)를 동시에 크롤링하고 결과를 합산. 속도가 중요한 파이프라인에 효과적입니다.

패턴 3: Sequential 패턴

에이전트들이 순서대로 실행되며 이전 결과를 다음 에이전트의 입력으로 사용합니다. 각 단계가 이전 결과에 의존할 때 적합합니다. 예: 문서 수집 → 요약 → 번역 → 리포트 생성. 가장 직관적인 패턴이지만 병목이 생기기 쉬우므로 각 단계 타임아웃 설정이 중요합니다.

어떤 패턴을 선택해야 하나

복잡한 의사결정이 필요한 조율: Supervisor. 독립 작업 병렬 처리: Parallel. 단계적 변환 파이프라인: Sequential. 실제 프로젝트에서는 이 세 패턴을 조합합니다. Supervisor가 일부 태스크는 Parallel로, 일부는 Sequential로 처리하도록 구성하는 것이 일반적입니다.